Paper number: D1343R1
Topic: Contracts Update for San Diego 2018
Author: Nathan Myers
Audience: EWG
Date: 2018-11-07

Contracts Update for San Diego 2018

The authors of the Contracts proposal met to discuss what changes might be needed to the WD text to address concerns raised on the e-mail reflector since Rapperswil.

The concerns break down, generally, into desire to control how compilers may use contract expressions in optimization, and how violation handlers behave on runs while one instruments legacy code.

Background

The design of the Contracts feature was driven by two principles:

1. The same library is often used in production, debugging, profiling, and whole-program analysis, built differently for each use. It is impractical to require changes to source text to adapt to each use.

2. The same contract annotation expression is useful to aid testing, to provide hints to optimizers, and to inform static analysis tooling.

For ease of discussion, we note that every contract annotation expression may be modeled as follows. The text

```
[[assert: expression]]
```

compiles, according to build mode, to the equivalent of either

```
{ if (!expression) std::unreachable(); }  // per P0627
```

(with no instructions actually emitted), or

```
{ if (!expression) __handle_violation(); }
```

(where `__handle_violation` calls the user's handler function and then either terminates or, if the user has so specified, continues execution).

Concerns

Reflector traffic has mostly concerned how users may achieve the degree of control they need to be able to use the feature well. These break down, roughly, into concerns about (1) limiting overreach by optimizers, and (2) staging instrumentation of existing code.

Optimization

Legitimate concerns were expressed that placing a contract annotation may cause a previously working program to fail as a result of optimizer activity:

In 9.11.4.3 Checking contracts [dcl.attr.contract.check] pp5:

> ... it is unspecified whether the predicate for a contract that is not checked under the current build level is evaluated; if the predicate of such a contract would evaluate to false, the behavior is

undefined.

There is a need to control such behavior at both program build time, and also in individual contract expressions.

Compilers always provide ways to control optimization at the TU level, so the need is for finer-grain control. We now know how to make normative text to control, at the TU level, use of the expressions for optimization, if necessary.

Fine-grained Control of Optimization

Sometimes the behavior of `<cassert>` assertions is closest to what users need, for a given expression. In an assert macro, if the expression is not evaluated, it has no effect on surrounding code. A simple workaround to get this effect, with the WD as-is, is by amending one's expression as follows:

```
[[assert: no() || expression]]
```

Evaluated at runtime, `no()` is a user-supplied function that returns false. When the expression is not evaluated, no definition need be provided for `no()` (or, equivalently, it may be defined to return true).

Then, the optimizer can draw no conclusions about the truth of the following expression. It cannot even assume the expression would have been safe to evaluate, so that (for example) pointer dereferences have no implication for preceding or subsequent code.

If needed this fine-grained control could be simply and directly supported in the core language. We do not have such a proposal here.

Fine-grained control of Resumption after Violation

Adding contract annotations to existing code typically reveals numerous latent violations with no evident effects. Identifying only a single such violation per build-test-fix cycle would be prohibitively slow. To identify violations wholesale, it is essential that program behavior be unchanged compared to the un-annotated code, beyond logging detected violations.

It turns out that, frequently, one only wants logging for violations originating in a particular subsystem under review. We observe that the user-supplied violation handler has full access to program global variables and to source location where the violation happened, so the caller may set flags (and function pointers) to influence its behavior.

Working Draft Changes

We suggest three minor changes to the WD.

1. Redundant normative text

In 9.11.4.3 Checking contracts [dcl.attr.contract.check] pp5 we have:

> "The violation handler of a program is a function ... and is specified in an implementation-defined manner."

The latter part of this quote is redundant with subsequent text:

> It is implementation-defined how the violation handler is established for a program

Suggest: Delete the redundant first part, starting at "and is".

2. Designating a violation handler

Both statements above fail to make clear that the violation handler is a function that may be *designated by the user*, nor what happens on a violation if the user does not designate a handler.

Suggest: The second part noted above should say

> It is implementation-defined how the violation handler for a program may be designated.

3. Side effect

The WD text allows an implementation to elide a returning handler that performs no side effects, along with expressions that would lead to calling it, if UB is encountered after the return.

Suggest: Prevent such elision by adding:

> "A contract violation is a side effect".

Add to 7.2.3 "Context dependence" [expr.context] paragraph 2:

> -- contract violation [dcl.attr.contract.check]